

Son Tekrar: Semaforlar (Semaphores) ve Monitörler (Monitors)

Süreç Senkronizasyonu konusunda öğrencilerin en çok karıştırdığı ve hocaların sınavda buradan eleme yapmayı en çok sevdiği iki kavramdır. Bu dokümanda aralarındaki farksal uçurumu kod örnekleriyle kapatıp, hocaların klasik "Tuzak Sorularını" deşifre edeceğiz.

1. Semafor (Semaphore) Nedir?

Hollandalı bilim insanı Dijkstra tarafından icat edilmiştir. Özünde, **yalnızca iki atomik komutla** (`wait` ve `signal` veya **P ve V**) erişilebilen ve değeri değiştirilebilen bir **Tamsayı (Integer)** değişkenidir (`S`).

- `wait(S)` (Bekle / Azalt):** Semaforun değerini 1 azaltır. Eğer değer 0 (veya negatif) ise prosesi uyutur ve bekleme kuyruğuna atar.
- `signal(S)` (Sinyal / Arttır):** Semaforun değerini 1 arttırır. Eğer kuyrukta uyuyan proses varsa **birini uyandırır** ve çalıştırır.

Semaforun Kanayan Yarası (Neden Sorunludur?)

Semafor işletim sistemi veya kütüphane düzeyindedir. Tüm kontrol ve sorumluluk **tamamen programcının (insanın) omuzlarındadır**.

Klasik Sınav Sorusu: "Semaforlarda Programcı Hatası" Hoca kod verir ve "Bu kodda ne hata var?" diye sorar. `C`

```
> // DOĞRU KULLANIM:  
> wait(mutex);  
> /* Kritik Bölge (CS) */  
> signal(mutex);  
>
```

Hata 1 (Sırayı Karıştırmak): `C`

```
> signal(mutex); // YANLIŞ! Kilit açma ile başladı  
> /* CS */  
> wait(mutex);  
>
```

Sonuç: Karşılıklı Dışlama (Mutual Exclusion) bozulur. Herkes içeri girer, Race Condition oluşur.

Hata 2 (Signal'i Unutmak / İki kere Wait yazmak): `C`

```
> wait(mutex);  
> /* CS */  
> wait(mutex); // YANLIŞ! Çıkarken de kilitledi  
>
```

Sonuç: Çıkışta kilit açılmadığı için kapıda bekleyen diğer prosesler sonsuza kadar bekler = **DEADLOCK (Ölümcül Kilitlenme)!**

2. Monitör (Monitor) Nedir?

Semaforlardaki "insan hatası" riskini (yanlış yere wait/signal yazıp sistemi çökertmeyi) ortadan kaldırmak için geliştirilmiş **Üst Düzey Programlama Dili (High-level Language) Yapısıdır**.

- Monitör, içerisinde kendi lokal değişkenlerini, fonksiyonlarını ve başlatma (init) kodlarını barındıran bir "Kutu" (Sınıf/Nesne) gibidir.
- Hayati Özelliği:** Monitörün içine aynı anda **SADECE BİR PROSES (Thread)** girebilir ve içerideki kodları çalıştırabilir. Bunu senin yerine **DERLEYİCİ (Dilin kendisi)** garanti eder!

Kod Örneği (Java - Monitör Kullanımı):

Java dilindeki `synchronized` anahtar kelimesi aslında bir monitördür. İşletim sistemi karmaşasıyla uğraşmazsın.

```
public class BankaHesabi {  
    private int bakiye = 100;  
  
    // "synchronized" kelimesi burayı bir Monitör yapar.  
    // İşletim sistemi, buraya aynı anda iki thread'in girmesini YASAKLAR.  
    public synchronized void paraCek(int miktar) {  
        if (bakiye >= miktar) {  
            bakiye = bakiye - miktar;  
        }  
    }  
}
```

Monitör İçinde Senkronizasyon (Condition Variables)

Sadece Mutual Exclusion (tek kişi girsin) yetmez, bazen giren kişinin sırası gelmemiştir ve beklemesi gerekir. Monitörler bunu **Condition (Koşul) Değişkenleri** ile çözer. Buralarda da `x.wait()` ve `x.signal()` kullanılır ama bu semaforlarınkinden farklıdır!

- `x.wait()`: Prosesi uyutur ve **monitörü hemen başka birine devreder** (kilit açılır).
- `x.signal()`: Uyuyan SADECE BİR prosesi anında uyandırıp sırayı ona verir.

3. SEMAFOR VE MONİTÖR KARŞILAŞTIRMASI (HOCANIN FAVORİ TABLOSU)

Özellik	Semafor (Semaphore)	Monitör (Monitor)
Seviye / Yapı	İşletim Sistemi / Kütüphane düzeyindedir. Basit bir tamsayıdır (Integer).	Derleyici / Programlama dili düzeyindedir (Java, C#). Gelişmiş bir Soyut Yapıdır (ADT).
Sorumluluk	Programcıya aittir. Wait/Signal sırasını programcı yönetir.	Derleyiciye (Sisteme) aittir. Kilit mekanizmasını dil kendi otomatik yönetir.
Hata İhtimali	Çok yüksektir. Kolayca Deadlock'a veya Race Condition'a sebep olunabilir.	Çok düşüktür. İnsan faktörü hatası minimize edilmiştir.
Mutual Exclusion	Wait ve Signal ile manuel sağlanmak zorundadır.	Monitör yapısı gereği Otomatik (varsayılan olarak) sağlanır.

4. HOCALARIN SORMAKTAN KEYİF ALDIĞI SINAV SORULARI

Soru 1: Semaforlar varken neden Monitörler icat edilmiştir / ihtiyaç duyulmuştur? Cevap: Semaforlarda Mutual Exclusion (Karşılıklı dışlama) sağlamak tamamen programcının `wait()` ve `signal()` komutlarını doğru ve hatasız yere yazmasına bağlıdır. Programcı yanlışlıkla sırayı karıştırırsa, bir `signal()` komutunu unutursa veya fazladan `wait()` yazarsa **Deadlock (Ölümcül kilitlenme)** veya **Yarış Durumu (Race Condition)** kaçınılmazdır. Monitörler, bu sorumluluğu programcıdan alıp **derleyiciye/dile** devrederek programcı kaynaklı hataları ve kilitlenmeleri önlemek, daha yüksek seviyeli/güvenli bir kodlama sunmak için icat edilmiştir.

Soru 2: İkili Semafor (Binary Semaphore) ile Mutex Lock arasındaki fark nedir? (Hocaların çok sevdiği tuzak sorulardan biridir, çünkü %90 ayındır denip geçilir ama ince bir farkı vardır.)
Cevap: İkisi de 0 ve 1 değeri alır ve Kritik bölgeyi korur. Ancak en büyük fark **Sahiplik (Ownership)** durumudur.

- Mutex'te** kilidi hangi proses aldıysa (kapattıysa), işi bitince kilidi **MECBUREN KENDİSİ** açmak zorundadır. Başkası gelip açamaz.
- İkili Semafor'da** böyle bir sahiplik kuralı yoktur. Bir proses `wait()` yapıp kilitlese bile, dışarıdan bambaşka bir proses gelip `signal()` göndererek o kilidi rastgele açabilir!

Soru 3: Monitör içerisindeki bir `x.signal()` komutu ile Semaforlardaki `signal(S)` komutunun farkı nedir? Cevap:

- Semaforda** kuyrukta uyuyan kimse yoksa (boşsa), `signal()` işlemi semafor değerini +1 artırır ve bu değer orada kalır (kaybolmaz, geleceğe yatırımdır).
- Monitörde** (Condition Variable'da) ise uyuyan kimse yokken `x.signal()` çağrılırsa bu sinyal boşluğa gider (**tamamen kaybolur/yok sayılır**). Değişkenin değerini kalıcı olarak etkilemez.

Soru 4: (Senaryo) 5 kişilik bir havuz verisi var. İşletim sistemi bu 5 kaynağı dinamik olarak yönetmek istiyor. Burada Mutex mi kullanılır, İkili (Binary) Semafor mu, Sayan

(Counting) Semafor mu? Cevap: Counting (Sayan) Semafor kullanılır. Çünkü Mutex ve Binary semafor SADECE 0 ve 1 değerini alır (tek bir kaynağı korur). 5 kaynak varsa Sayan Semafor ilk değerini 5 yapar, her `wait()` işleminde 1 azalır, 0 olduğunda yeni gelen 6. kişiyi içeri almaz uyutur. Kaynak havuzları (Örn: 5 yazıcı bağlantısı) sayan semaforlarla yönetilir.